

GUÍA DE AUTOAPRENDIZAJE

Dockerización y Despliegue de Aplicaciones Spring Boot en Microsoft Azure

Contenedores, base de datos PostgreSQL, CI/CD, gestión de secretos y monitoreo en la nube

Autor

Dr. Lino Martin Quispe Tincopa

8 de julio de 2026

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción	3
1.1 Objetivos de aprendizaje	3
2. Opciones de Despliegue de Contenedores en Azure	3
3. Requisitos Previos	3
4. Dockerización de la Aplicación Spring Boot	4
4.1 Prueba local.....	4
5. Azure Container Registry (ACR)	4
6. Azure Database for PostgreSQL Flexible Server.....	4
6.1 Dependencia y configuración en Spring Boot.....	5
7. Despliegue en Azure App Service.....	5
8. Integración y Despliegue Continuo con GitHub Actions	6
8.1 Service Principal para autenticación	6
8.2 Secrets requeridos en GitHub	6
8.3 Workflow (.github/workflows/deploy.yml)	6
9. Gestión Segura de Secretos con Azure Key Vault	7
10. Monitoreo con Application Insights.....	8
10.1 Alertas básicas.....	8
11. Arquitectura Final	8
11.1 Resumen de componentes implementados	9
12. Próximos Pasos Sugeridos.....	9

1. Introducción

Esta guía documenta el proceso completo para contenedorizar una aplicación desarrollada en Spring Boot y desplegarla en Microsoft Azure utilizando buenas prácticas de la industria: contenedores Docker, base de datos gestionada, integración continua, gestión segura de secretos y observabilidad en producción.

Está pensada como material de autoestudio: cada sección incluye explicación conceptual, comandos ejecutables y notas de advertencia sobre errores comunes.

1.1 Objetivos de aprendizaje

- Comprender las opciones de despliegue de contenedores en Azure y cuándo usar cada una.
- Construir una imagen Docker optimizada para una aplicación Spring Boot.
- Desplegar la aplicación en Azure App Service usando Azure Container Registry.
- Aprovisionar y conectar una base de datos Azure Database for PostgreSQL.
- Automatizar el ciclo de build y despliegue con GitHub Actions.
- Proteger credenciales sensibles usando Azure Key Vault.
- Monitorear la aplicación en producción con Application Insights.

2. Opciones de Despliegue de Contenedores en Azure

Azure ofrece varios servicios para ejecutar contenedores. Elegir el correcto depende de la complejidad de la aplicación y del nivel de control que se necesite.

Servicio	Cuándo usarlo
Azure Container Instances (ACI)	Contenedores sueltos, rápidos, sin orquestación; ideal para pruebas o tareas puntuales.
Azure App Service (Web App for Containers)	Apps web/API en contenedor, con escalado sencillo, SSL y dominios; usada en esta guía.
Azure Container Apps	Microservicios y apps event-driven, con autoscaling basado en KEDA.
Azure Kubernetes Service (AKS)	Cuando se necesita Kubernetes real, con múltiples servicios y alta complejidad.

Para la mayoría de aplicaciones web como la de esta guía, la combinación más práctica es Azure Container Registry (ACR) junto con Azure App Service.

3. Requisitos Previos

Antes de comenzar, es necesario tener instalado Azure CLI y una cuenta activa de Azure.

```
# Instalar Azure CLI
curl -L https://aka.ms/InstallAzureCLI | bash

# Iniciar sesión
az login
```

```
# Crear un grupo de recursos
az group create --name mi-grupo-rg --location eastus
```

4. Dockerización de la Aplicación Spring Boot

Se utiliza una imagen multi-stage: una etapa de construcción con Maven y una etapa final ligera basada en el runtime JRE de Eclipse Temurin.

```
# --- Etapa de construcción ---
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# --- Etapa de ejecución ---
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Nota: ajustar la versión de Java (17) según la definida en el pom.xml del proyecto.

4.1 Prueba local

```
docker build -t mi-app-springboot:v1 .
docker run -p 8080:8080 mi-app-springboot:v1
```

5. Azure Container Registry (ACR)

El ACR almacena las imágenes Docker de forma privada dentro de Azure.

```
az acr create --resource-group mi-grupo-rg \
  --name miregistroacr \
  --sku Basic

# Construir la imagen directamente en Azure (sin build local)
az acr build --registry miregistroacr --image spring-app:v1 .
```

6. Azure Database for PostgreSQL Flexible Server

Se aprovisiona un servidor PostgreSQL gestionado por Azure, incluyendo la base de datos de la aplicación y las reglas de firewall necesarias.

```
az postgres flexible-server create \
  --resource-group mi-grupo-rg \
  --name mi-postgres-server \
  --location eastus \
  --admin-user adminuser \
  --admin-password 'TuPasswordSegura123!' \
  --sku-name Standard_B1ms \
  --tier Burstable \
```

```
--storage-size 32 \  
--version 16 \  
--public-access 0.0.0.0  
  
az postgres flexible-server db create \  
--resource-group mi-grupo-rg \  
--server-name mi-postgres-server \  
--database-name mi_app_db  
  
az postgres flexible-server firewall-rule create \  
--resource-group mi-grupo-rg \  
--name mi-postgres-server \  
--rule-name AllowAzureServices \  
--start-ip-address 0.0.0.0 \  
--end-ip-address 0.0.0.0
```

Nota: para producción, se recomienda usar integración VNet entre App Service y PostgreSQL en lugar de abrir el firewall a todos los servicios de Azure.

6.1 Dependencia y configuración en Spring Boot

```
<dependency>  
  <groupId>org.postgresql</groupId>  
  <artifactId>postgresql</artifactId>  
  <scope>runtime</scope>  
</dependency>  
  
spring.datasource.url=${SPRING_DATASOURCE_URL}  
spring.datasource.username=${SPRING_DATASOURCE_USERNAME}  
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD}  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

7. Despliegue en Azure App Service

```
az appservice plan create --name plan-springboot \  
--resource-group mi-grupo-rg \  
--is-linux --sku B1  
  
az webapp create --resource-group mi-grupo-rg \  
--plan plan-springboot \  
--name mi-app-springboot-unica \  
--deployment-container-image-name miregistroacr.azurecr.io/spring-app:v1  
  
az webapp identity assign --name mi-app-springboot-unica --resource-group mi-grupo-rg  
  
principalId=$(az webapp identity show --name mi-app-springboot-unica \  
--resource-group mi-grupo-rg --query principalId -o tsv)  
acrId=$(az acr show --name miregistroacr --query id -o tsv)  
az role assignment create --assignee $principalId --scope $acrId --role AcrPull  
  
az webapp config container set --name mi-app-springboot-unica \  
--resource-group mi-grupo-rg \  
--docker-custom-image-name miregistroacr.azurecr.io/spring-app:v1 \  
--docker-registry-server-url https://miregistroacr.azurecr.io  
  
az webapp config appsettings set --resource-group mi-grupo-rg \  
--name mi-app-springboot-unica --setting-name SPRING_DATASOURCE_URL --value jdbc:postgresql://mi-postgres-server.mi-grupo-rg.database.azure.com:5432/mi_app_db
```

```
--name mi-app-springboot-unica \  
--settings WEBSITES_PORT=8080
```

Variables de conexión a la base de datos:

```
az webapp config appsettings set --resource-group mi-grupo-rg \  
  --name mi-app-springboot-unica \  
  --settings \  
    SPRING_DATASOURCE_URL="jdbc:postgresql://mi-postgres-  
server.postgres.database.azure.com:5432/mi_app_db?sslmode=require" \  
    SPRING_DATASOURCE_USERNAME="adminuser" \  
    SPRING_DATASOURCE_PASSWORD="TuPasswordSegura123!"
```

Verificación de logs en tiempo real:

```
az webapp log tail --name mi-app-springboot-unica --resource-group mi-grupo-rg
```

8. Integración y Despliegue Continuo con GitHub Actions

La automatización permite que cada push a la rama principal dispare el build de la imagen, su publicación en ACR y la actualización del App Service, sin intervención manual.

8.1 Service Principal para autenticación

```
az ad sp create-for-rbac \  
  --name "github-actions-springboot" \  
  --role contributor \  
  --scopes /subscriptions/<SUBSCRIPTION_ID>/resourceGroups/mi-grupo-rg \  
  --sdk-auth
```

8.2 Secrets requeridos en GitHub

- AZURE_CREDENTIALS — JSON completo del service principal
- ACR_NAME, ACR_LOGIN_SERVER — nombre y servidor del registro
- ACR_USERNAME, ACR_PASSWORD — credenciales del ACR (az acr credential show)
- WEBAPP_NAME, RESOURCE_GROUP — nombre de la app y del grupo de recursos

8.3 Workflow (.github/workflows/deploy.yml)

```
name: Build and Deploy Spring Boot to Azure  
  
on:  
  push:  
    branches:  
      - main  
  
env:  
  IMAGE_NAME: spring-app  
  
jobs:  
  build-and-deploy:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4
```

```
- uses: azure/docker-login@v1
  with:
    login-server: ${ secrets.ACR_LOGIN_SERVER }}
    username: ${ secrets.ACR_USERNAME }}
    password: ${ secrets.ACR_PASSWORD }}

- run: |
  docker build -t ${ secrets.ACR_LOGIN_SERVER }}/${ env.IMAGE_NAME }}:${ secrets.github.sha
}} .
  docker push ${ secrets.ACR_LOGIN_SERVER }}/${ env.IMAGE_NAME }}:${ secrets.github.sha }}

- uses: azure/login@v2
  with:
    creds: ${ secrets.AZURE_CREDENTIALS }}

- uses: azure/webapps-deploy@v3
  with:
    app-name: ${ secrets.WEBAPP_NAME }}
    images: ${ secrets.ACR_LOGIN_SERVER }}/${ env.IMAGE_NAME }}:${ secrets.github.sha }}
```

Nota: para mayor seguridad, se puede reemplazar el uso de secrets estáticos por autenticación OIDC federada entre GitHub y Azure.

9. Gestión Segura de Secretos con Azure Key Vault

En lugar de almacenar contraseñas en texto plano dentro de la configuración del App Service, se centralizan en Key Vault y se referencian de forma segura.

```
az keyvault create \
  --name mi-keyvault-app \
  --resource-group mi-grupo-rg \
  --location eastus

az keyvault secret set --vault-name mi-keyvault-app \
  --name "SPRING-DATASOURCE-URL" \
  --value "jdbc:postgresql://mi-postgres-
server.postgres.database.azure.com:5432/mi_app_db?sslmode=require"

az keyvault secret set --vault-name mi-keyvault-app \
  --name "SPRING-DATASOURCE-PASSWORD" \
  --value "TuPasswordSegura123!"

principalId=$(az webapp identity show --name mi-app-springboot-unica \
  --resource-group mi-grupo-rg --query principalId -o tsv)

az keyvault set-policy \
  --name mi-keyvault-app \
  --object-id $principalId \
  --secret-permissions get list

az webapp config appsettings set \
  --resource-group mi-grupo-rg \
  --name mi-app-springboot-unica \
  --settings \
    SPRING_DATASOURCE_PASSWORD="@Microsoft.KeyVault(SecretUri=https://mi-keyvault-
app.vault.azure.net/secrets/SPRING-DATASOURCE-PASSWORD/)"
```

App Service resuelve automáticamente la referencia en tiempo de ejecución usando su identidad administrada; el valor real nunca queda expuesto en configuración ni en logs.

10. Monitoreo con Application Insights

```
az monitor app-insights component create \  
  --app mi-app-insights \  
  --location eastus \  
  --resource-group mi-grupo-rg \  
  --application-type java
```

El agente Java de Application Insights se añade a la imagen sin modificar el código de la aplicación:

```
FROM eclipse-temurin:17-jre-alpine  
WORKDIR /app  
ADD https://github.com/microsoft/ApplicationInsights-  
Java/releases/latest/download/applicationinsights-agent.jar /app/applicationinsights-agent.jar  
COPY --from=build /app/target/*.jar app.jar  
EXPOSE 8080  
ENTRYPOINT ["java", "-javaagent:/app/applicationinsights-agent.jar", "-jar", "app.jar"]  
  
CONNECTION_STRING=$(az monitor app-insights component show \  
  --app mi-app-insights --resource-group mi-grupo-rg --query connectionString -o tsv)  
  
az webapp config appsettings set --resource-group mi-grupo-rg \  
  --name mi-app-springboot-unica \  
  --settings APPLICATIONINSIGHTS_CONNECTION_STRING="$CONNECTION_STRING"
```

10.1 Alertas básicas

```
az monitor metrics alert create \  
  --name "alerta-latencia-alta" \  
  --resource-group mi-grupo-rg \  
  --scopes $(az monitor app-insights component show --app mi-app-insights --resource-group mi-  
grupo-rg --query id -o tsv) \  
  --condition "avg requests/duration > 2000" \  
  --evaluation-frequency 5m --window-size 15m --severity 2
```

Desde el portal de Azure, el recurso de Application Insights ofrece sin configuración adicional: Application Map (dependencias), Live Metrics (tráfico en tiempo real), Failures (excepciones agrupadas) y Performance (tiempos de respuesta por endpoint).

11. Arquitectura Final

El flujo completo de la solución queda representado de la siguiente manera:

```
GitHub (push a main)  
|  
GitHub Actions (build + push imagen)  
|  
Azure Container Registry (ACR)  
|  
Azure App Service (Spring Boot) <---> Azure Key Vault (secrets)  
|  
|
```


Azure Database for PostgreSQL Flexible Server	Application Insights (metricas + alertas)
--	--

11.1 Resumen de componentes implementados

- Contenedorización de la aplicación Spring Boot con Docker multi-stage.
- Registro y versionado de imágenes en Azure Container Registry.
- Despliegue en Azure App Service para contenedores Linux.
- Base de datos gestionada Azure Database for PostgreSQL Flexible Server.
- Integración y despliegue continuo automatizado con GitHub Actions.
- Gestión centralizada y segura de credenciales con Azure Key Vault.
- Observabilidad y alertas en producción con Application Insights.

12. Próximos Pasos Sugeridos

- Configurar un dominio personalizado con certificado SSL para el App Service.
- Crear un ambiente de staging separado antes de producción (slots de despliegue).
- Migrar la autenticación de GitHub Actions a OIDC federado, eliminando secrets estáticos.
- Implementar integración VNet entre App Service y PostgreSQL para mayor aislamiento de red.
- Explorar Azure Container Apps si la arquitectura evoluciona hacia microservicios.